

Technical Analysis of a Statistical Pattern Recognition Implementation

Perceptron, Least-Squares, and Multicategory Logistic Discrimination

Source-faithful technical report based on the supplied Python implementation.

Contents

1	Introduction	3
2	Project Overview	3
3	Technical Foundations	4
3.1	Binary linear discrimination	4
3.2	Perceptron learning	4
3.3	Least-squares classification	5
3.4	Binary logistic discrimination	5
4	Data and Input Processing	6
4.1	Binary subset construction	6
4.2	Bias augmentation	6
5	System Architecture and Code Organization	6
6	Detailed Methodology	6
6.1	Perceptron workflow	6
6.2	Least-squares workflow	7
6.3	Logistic core implementation	8
6.4	One-vs-rest logistic discrimination	8
6.5	One-vs-one logistic discrimination	9
7	Mathematical Formulation and Implementation Traceability	9
8	Optimization and Learning Procedure	9
9	Implementation Details	10
9.1	Dependencies	10
9.2	Determinism	10
9.3	Filesystem behavior	10
9.4	Serialization	10
10	Execution Workflow	11
11	Design Decisions and Rationale	11
12	Computational Considerations	11
13	Reproducibility and Reliability	12
14	Limitations and Technical Considerations	12
14.1	No probabilistically normalized multiclass logistic model	12
14.2	Pairwise tie-breaking	12
14.3	Logistic stopping criterion	13
14.4	Stored logistic loss	13
14.5	Hyperparameter dependence	13
14.6	Scope of validation	13
15	Overall Technical Assessment	13

16 Conclusion	14
A Source-to-Report Traceability Map	14
B Representative Code Excerpts	14
B.1 Perceptron update	15
B.2 Least-squares solution	15
B.3 Logistic gradient	15

Abstract

This report analyzes the supplied Python implementation for Statistical Pattern Recognition Computer Assignment 3. The program constructs the assignment’s four-class, two-dimensional training set and implements three families of discriminative methods: binary perceptron classifiers for two required class pairs, linear least-squares classification using the Moore–Penrose pseudoinverse, and binary logistic discriminants composed into multiclass one-vs-rest and one-vs-one systems. The analysis covers the software structure, data representation, mathematical formulation, optimization procedures, numerical safeguards, execution workflow, artifact generation, and reproducibility mechanisms. It focuses in particular on the differences between the textbook methods and their concrete implementation, including the perceptron’s dependence on sample order, the minimum-norm least-squares solution produced by the pseudoinverse, and the absence of a finite unregularized logistic optimum for separable data. It also records implementation-level limitations supported by the source, including the interpretation of the independently fitted one-vs-rest sigmoid outputs, one-vs-one vote ties, the exact logistic stopping test and stored loss value, and the ordering issue in the final artifact manifest. No experimental figures or result visualizations are reproduced; the emphasis is on the computational method and its implementation.

1 Introduction

The supplied program is a complete script-style Python implementation of a classical statistical pattern recognition assignment. Its computational scope is divided into three main components: binary perceptron learning, linear classification under a least-squares criterion, and multiclass logistic discrimination. The implementation also includes supporting infrastructure for deterministic configuration, dataset serialization, result tables, ambiguity analysis, and artifact packaging.

The project uses a low-dimensional supervised classification setting. Each observation has two numerical features, x_1 and x_2 , and belongs to one of four classes, denoted $\omega_1, \omega_2, \omega_3, \omega_4$. The program reduces several multiclass problems to binary decisions, trains the requested discriminants, and stores structured outputs in CSV form. It also creates plots during execution, but those outputs are outside the scope of this report.

The report does not reproduce the source line by line. Instead, it reconstructs the program’s pipeline and explains how its functions, data structures, mathematical operations, control flow, and output logic fit together. Differences from an idealized textbook treatment are identified explicitly. When author intent cannot be established from the code, the discussion is limited to what can be inferred from the implementation.

2 Project Overview

Execution proceeds as follows:

1. Establish a deterministic numerical environment and import the required scientific-Python packages.
2. Materialize the four-class assignment dataset as a `pandas.DataFrame` and save an exact CSV copy.
3. Provide helper functions for bias augmentation and extraction of binary class pairs.
4. Train perceptron models for ω_1 versus ω_2 and ω_3 versus ω_2 , record convergence information, and serialize the resulting parameters.
5. Solve the same two binary class problems using a least-squares criterion and the Moore–Penrose pseudoinverse.

6. Implement an unregularized binary logistic learner using a sigmoid function and batch gradient descent.
7. Build one-vs-rest logistic models, use their scores for multiclass prediction, and characterize potentially ambiguous regions by two explicit probability-based conditions.
8. Build one-vs-one logistic models for all $\binom{4}{2} = 6$ unordered class pairs, combine their decisions through majority voting, and flag vote ties.
9. Consolidate selected metrics into a summary CSV and package generated artifacts into a ZIP archive, with an optional Google Colab download action.

The program is self-contained. The learning algorithms are implemented directly with NumPy rather than through a machine-learning library, so the main computational steps—matrix products, vector updates, gradients, thresholding, voting, and stopping conditions—are explicit in the source.

3 Technical Foundations

3.1 Binary linear discrimination

All three primary methods ultimately use an affine discriminant in the original two-dimensional feature space. With the augmented vector

$$\tilde{\mathbf{x}} = \begin{bmatrix} x_1 \\ x_2 \\ 1 \end{bmatrix}, \quad (1)$$

the parameter vector

$$\mathbf{w} = \begin{bmatrix} w_1 \\ w_2 \\ b \end{bmatrix} \quad (2)$$

produces the scalar score

$$s(\mathbf{x}) = \mathbf{w}^T \tilde{\mathbf{x}} = w_1 x_1 + w_2 x_2 + b. \quad (3)$$

The associated decision boundary is the set of points for which $s(\mathbf{x}) = 0$. In two dimensions, this boundary is a line. When the coefficient of x_2 is numerically zero, the same boundary is represented as a vertical line.

For binary encodings with labels $y_i \in \{-1, +1\}$, the sign of $s(\mathbf{x}_i)$ determines the predicted class. For the logistic model, the same affine score is mapped to $[0, 1]$ through the sigmoid function.

3.2 Perceptron learning

The perceptron implementation uses the standard signed-margin test

$$y_i \mathbf{w}^T \tilde{\mathbf{x}}_i \leq 0 \quad (4)$$

to identify a misclassified or boundary-touching sample. When the condition holds, the parameter vector is updated as

$$\mathbf{w} \leftarrow \mathbf{w} + \eta y_i \tilde{\mathbf{x}}_i, \quad (5)$$

where η is the learning rate. The source initializes $\mathbf{w}$ to zero and uses $\eta = 1$ by default.

The code counts complete passes through the training set as epochs and stops when an entire pass contains no mistakes. Consequently, the reported “iterations to convergence” field is

an epoch count, not the number of individual weight updates. The implementation uses the fixed order of the samples in the constructed DataFrame, which matters because perceptron updates are sequential and order-dependent even when a linearly separable dataset guarantees convergence under standard assumptions.

3.3 Least-squares classification

For least-squares classification, the binary target vector is encoded with $+1$ and -1 . The affine model can be written as

$$\hat{\mathbf{y}} = \mathbf{X}\mathbf{w}, \quad (6)$$

where $\mathbf{X}$ is the bias-augmented design matrix. The optimization problem is

$$\min_{\mathbf{w}} \|\mathbf{X}\mathbf{w} - \mathbf{y}\|_2^2. \quad (7)$$

Rather than explicitly forming a matrix inverse, the program computes

$$\mathbf{w} = \mathbf{X}^+ \mathbf{y}, \quad (8)$$

where $\mathbf{X}^+$ denotes the Moore–Penrose pseudoinverse. NumPy’s `pinv` produces the minimum-norm least-squares solution when the system does not have a unique solution. Classification then uses the sign of the linear score.

This method should be distinguished from probabilistic logistic regression. The least-squares objective minimizes squared residuals against the chosen ± 1 targets; it does not model class probabilities directly.

3.4 Binary logistic discrimination

The source implements the sigmoid function

$$\sigma(z) = \frac{1}{1 + e^{-z}}. \quad (9)$$

For a sample $\tilde{\mathbf{x}}_i$, the model probability is

$$p_i = \sigma(\mathbf{w}^\top \tilde{\mathbf{x}}_i). \quad (10)$$

The binary cross-entropy objective used by the code is

$$\mathcal{L}(\mathbf{w}) = -\frac{1}{N} \sum_{i=1}^N [y_i \log p_i + (1 - y_i) \log(1 - p_i)], \quad (11)$$

with $y_i \in \{0, 1\}$. Its batch gradient is

$$\nabla_{\mathbf{w}} \mathcal{L} = \frac{1}{N} \mathbf{X}^\top (\mathbf{p} - \mathbf{y}), \quad (12)$$

which leads to the explicit update

$$\mathbf{w}_{t+1} = \mathbf{w}_t - \eta \nabla_{\mathbf{w}} \mathcal{L}(\mathbf{w}_t). \quad (13)$$

The implementation does not use regularization. On separable data, a linear separator can classify the samples perfectly while the logistic likelihood continues to improve as the parameter norm grows, so a finite maximum-likelihood solution need not exist. The optimizer is therefore bounded by a fixed iteration budget and returns an explicit Boolean `converged` flag.

4 Data and Input Processing

The dataset is embedded directly in the source as the `CLASS_DATA` dictionary. Four class names are used as keys, and each key maps to ten two-dimensional coordinate pairs. The program then iterates through this nested structure and creates one tabular row per sample containing the class label, a one-based sample identifier, and the two features `x1` and `x2`. The resulting DataFrame therefore contains 40 observations, four class labels, and two numerical predictor columns.

The dataset is constructed deterministically and does not depend on a network download or an external dataset package. The source uses the assignment's own training table as the input. The DataFrame is immediately serialized to `assignment3_dataset.csv`, making the exact in-code dataset easy to inspect independently.

The program also contains a dataset-visualization block, but it is a presentation artifact rather than a dependency of the learning algorithms. The classifiers operate directly on the DataFrame and its NumPy representations, so the report does not reproduce or rely on those plots.

4.1 Binary subset construction

The helper `get_binary_pair(c1,c2)` filters the DataFrame to two requested classes, extracts x_1 and x_2 as floating-point arrays, and encodes the first requested class as $+1$ and the second as -1 . This helper establishes a common binary representation shared by the perceptron and least-squares experiments.

4.2 Bias augmentation

The helper `add_bias` appends a column of ones to an input feature matrix. For N observations, this transforms an $N \times 2$ matrix into an $N \times 3$ matrix. The third coefficient in $\mathbf{w}$ therefore acts as an intercept without requiring a separate special case in the linear algebra.

5 System Architecture and Code Organization

The program is a single, sequential Python module rather than a multi-file package. Nevertheless, its functions form a clear internal architecture.

The execution order is straightforward: data first, then the binary methods, then the logistic building blocks, followed by the multicategory compositions and finally the summary/export stages. Functions are defined before they are called, so the script can run from top to bottom without relying on hidden notebook state.

The supplied file is ordinary Python code with notebook-style documentation strings used to describe sections and cells. It therefore has the structure of a script extracted from a Colab notebook rather than a separate notebook JSON file.

6 Detailed Methodology

6.1 Perceptron workflow

For each required class pair, the program performs the following pipeline:

1. Filter the full DataFrame to the two requested classes.
2. Encode the first class as $+1$ and the second as -1 .

Table 1: Functional decomposition of the implementation.

Component	Representative functions	Responsibility
Data and utilities	<code>add_bias</code> ,	Data reshaping, binary label construction, bias augmentation.
Perceptron	<code>get_binary_pair</code> , <code>perceptron_train</code> , <code>plot_binary_boundary</code>	Sequential online updates, convergence accounting, boundary serialization.
Least squares	<code>least_squares_train</code>	Pseudoinverse solution, score calculation, thresholding, accuracy computation.
Logistic core	<code>sigmoid</code> , <code>logistic_train</code>	Stable sigmoid evaluation, cross-entropy gradient descent, stopping logic.
One-vs-rest	<code>train_one_vs_rest</code> , <code>predict_one_vs_rest</code>	Four binary classifiers, score collection, class selection, ambiguity analysis.
One-vs-one	<code>train_one_vs_one</code> , <code>pairwise_probability</code> , <code>predict_one_vs_one</code>	Six pairwise logistic models, probability orientation, majority voting, tie flags.
Artifact layer	summary and export cells	CSV serialization, manifest creation, ZIP packaging, optional Colab download.

3. Append a bias coordinate.
4. Initialize the three-dimensional parameter vector to zero.
5. Iterate through the samples in fixed DataFrame order.
6. Apply the perceptron update to every sample satisfying the signed-margin condition.
7. Count mistakes for the completed pass.
8. Stop when the mistake count becomes zero, or stop after the maximum epoch budget.

The source returns the learned weights, the stopping epoch, the list of per-epoch mistake counts, and the convergence status. The caller stores the most recent mistake count as a diagnostic field and writes a single CSV row per class pair.

The boundary plot helper converts the learned affine equation into an explicit x_2 -as-a-function-of- x_1 expression whenever $w_2 \neq 0$:

$$x_2 = -\frac{w_1 x_1 + b}{w_2}. \quad (14)$$

If w_2 is numerically negligible, the code instead draws the vertical boundary $x_1 = -b/w_1$. This is a numerical safeguard for the two-dimensional visualization logic; the visualization itself is not needed for prediction.

6.2 Least-squares workflow

The least-squares experiment reuses the binary-pair helper and bias augmentation. For each requested pair, `least_squares_train` computes the pseudoinverse solution, obtains the continuous score vector $\mathbf{X}\mathbf{w}$, thresholds the score at zero, and measures the fraction of correctly classified training samples. The output record stores the two feature coefficients, the bias, and the resulting training accuracy.

The code also avoids implementing the normal equations by hand. Using `np.linalg.pinv` avoids an explicit inverse of $\mathbf{X}^\top \mathbf{X}$ and handles rank-deficient systems more gracefully than a direct matrix inversion.

6.3 Logistic core implementation

The logistic learner starts from a zero parameter vector for every binary problem. On each iteration it:

1. evaluates $\mathbf{p} = \sigma(\mathbf{X}\mathbf{w})$;
2. forms the batch gradient $\mathbf{X}^\top(\mathbf{p} - \mathbf{y})/N$;
3. proposes the candidate update $\mathbf{w}' = \mathbf{w} - \eta \nabla \mathcal{L}$;
4. computes the current cross-entropy loss using clipped probabilities;
5. checks whether $\|\mathbf{w}' - \mathbf{w}\|_2$ is at most the tolerance;
6. returns the candidate and convergence flag when the stopping test passes; otherwise assigns the candidate to $\mathbf{w}$ and continues.

The sigmoid implementation clips its input to $[-60, 60]$ before evaluating e^{-z} . This is a numerical-stability measure: extremely large positive or negative logits can otherwise create avoidable overflow or underflow warnings. The loss computation separately clips probabilities to $[10^{-12}, 1 - 10^{-12}]$ before applying logarithms, preventing $\log(0)$ from being evaluated.

One implementation detail should be preserved in any future refactor. The returned `loss` value is computed from p associated with the parameter vector before the candidate update, while the function may return the candidate vector when the step-size test succeeds. Thus, the CSV field `final_loss` represents the loss immediately before the accepted stopping update, not a recomputation at the returned candidate. The value is a valid diagnostic, but the field name may suggest otherwise.

6.4 One-vs-rest logistic discrimination

The one-vs-rest routine trains four independent binary models. For a target class ω_k , the label is one when the sample belongs to ω_k and zero otherwise. The resulting probability-like score is

$$p_k(\mathbf{x}) = \sigma(\mathbf{w}_k^\top \tilde{\mathbf{x}}). \quad (15)$$

For multiclass prediction, the implementation chooses the class with the maximum of the four scores:

$$\hat{y} = \arg \max_k p_k(\mathbf{x}). \quad (16)$$

The four sigmoid outputs are fitted independently and are not constrained to sum to one. They therefore should not be interpreted as a normalized four-class posterior vector. The argmax operation is still a valid decision rule for the implementation, but “independent binary probabilities” should be distinguished from “a single normalized multiclass probability model.”

The ambiguity analysis creates a dense rectangular grid over the observed feature range with a fixed margin. For each grid point it computes the four scores, sorts them, and flags the point when either the largest score is below 0.60 or the gap between the largest and second-largest score is below 0.10:

$$\text{ambiguous}(\mathbf{x}) = \left[p_{(1)}(\mathbf{x}) < 0.60 \right] \vee \left[p_{(1)}(\mathbf{x}) - p_{(2)}(\mathbf{x}) < 0.10 \right]. \quad (17)$$

Here $p_{(1)}$ and $p_{(2)}$ denote the largest and second-largest scores, respectively. The first condition represents weak absolute support; the second represents weak relative separation between the two leading classes.

6.5 One-vs-one logistic discrimination

The one-vs-one routine forms all unordered class pairs using `itertools.combinations`. With four classes, there are six pairwise models. Within each pair, the lexicographically first class is encoded as one and the other as zero. The helper `pairwise_probability` then reorients the returned probability when the caller asks about the opposite class order.

For an input point, every pairwise model casts exactly one vote. The resulting vote matrix has one column per class. The final decision is

$$\hat{y} = \arg \max_k v_k(\mathbf{x}), \quad (18)$$

where $v_k(\mathbf{x})$ is the number of pairwise models favoring class k .

The source also computes a Boolean tie indicator whenever more than one class attains the maximum vote count. The prediction itself still comes from NumPy's `argmax`, which selects the first maximum according to the class ordering. Therefore, a vote tie produces a deterministic tie-break rather than a unique winner, and the accompanying `ovo_tie` flag is needed to interpret such cases correctly.

7 Mathematical Formulation and Implementation Traceability

This section collects the core equations and connects them directly to the source functions.

Table 2: Core mathematical operations and their implementation roles.

Expression	Implementation	ele-	Meaning in the program
$\tilde{\mathbf{x}} = [x_1, x_2, 1]^\top$	<code>add_bias</code>		Adds the intercept coordinate.
$y_i \mathbf{w}^\top \tilde{\mathbf{x}}_i \leq 0$	<code>perceptron_train</code>		Identifies an update-triggering sample.
$\mathbf{w} \leftarrow \mathbf{w} + \eta y_i \tilde{\mathbf{x}}_i$	<code>perceptron_train</code>		Performs the perceptron update.
$\mathbf{w} = \mathbf{X}^+ \mathbf{y}$	<code>least_squares_train</code>		Computes the least-squares solution via pseudoinverse.
$\sigma(z) = 1/(1 + e^{-z})$	<code>sigmoid</code>		Maps a linear score to a binary probability-like quantity.
$\nabla \mathcal{L} = \mathbf{X}^\top (\mathbf{p} - \mathbf{y})/N$	<code>logistic_train</code>		Computes the batch logistic gradient.
$\mathbf{w}' = \mathbf{w} - \eta \nabla \mathcal{L}$	<code>logistic_train</code>		Applies one gradient-descent step.
$\arg \max_k p_k$	<code>predict_one_vs_rest</code>		Converts four independent scores into a class decision.
$\arg \max_k v_k$	<code>predict_one_vs_one</code>		Selects the class with the most pairwise votes.

The mathematical structure is internally coherent: a common affine representation supports all binary learners, while each method changes the objective or decision rule applied to that representation. This shared structure is one reason the code can reuse the same `add_bias` and class-pair preparation logic.

8 Optimization and Learning Procedure

The perceptron is not formulated in the program as an explicit smooth optimization problem. Its stopping condition is combinatorial: an epoch is successful when it contains zero update-

triggering samples. The maximum epoch limit provides a finite safeguard if that condition is not achieved.

Least-squares learning is closed-form in the implementation. There is no iterative optimizer, learning rate, or tolerance. The computational bottleneck is the pseudoinverse computation performed by NumPy.

Logistic learning is the only explicitly iterative continuous optimization routine. The implementation uses full-batch gradient descent with the fixed defaults $\eta = 0.25$, a maximum of 10,000 iterations, and a parameter-step tolerance of 10^{-8} . Because no learning-rate schedule, line search, Newton method, or regularization term is present, the exact convergence behavior depends on the scale and geometry of each binary problem.

The treatment of separability is important. In a separable logistic problem, reducing the loss can require increasingly large parameter magnitudes, so a parameter-step criterion combined with a hard iteration budget may terminate without reaching a finite optimum. Recording `converged=False` in this situation is preferable to reporting success.

9 Implementation Details

9.1 Dependencies

The implementation relies on standard Python and four principal scientific-computing packages: NumPy for numerical arrays and linear algebra, pandas for tabular data, Matplotlib for visualization and file output, and IPython for notebook-friendly display. The script also contains an optional Google Colab import used only in the final download step.

The standard-library dependencies have distinct roles. `Path` handles filesystem paths, `combinations` enumerates class pairs, `zipfile` creates the archive, and `shutil` copies an optional standalone source script. `sys` provides version reporting. The imported `math` module is not used by the supplied implementation.

9.2 Determinism

The program defines `SEED = 42` and calls `np.random.seed(SEED)`. This establishes a deterministic NumPy random state, although the actual algorithms shown in the source do not use random sampling. The important sources of determinism are therefore primarily fixed input order, fixed numerical hyperparameters, and deterministic NumPy operations rather than stochastic training.

9.3 Filesystem behavior

The output root is computed as `Path.cwd()` followed by an `assignment3_outputs` subdirectory. This means that artifact location depends on the current working directory. The behavior is convenient in Colab or a controlled script environment, but it is not intrinsically independent of launch location.

9.4 Serialization

Each major experiment writes a dedicated CSV. This is useful for downstream auditing because learned parameters and diagnostic fields are stored as tabular artifacts rather than existing only in console output. The final summary CSV provides a compact cross-method index of selected quantities.

10 Execution Workflow

The source is organized as a top-to-bottom executable pipeline. The initial environment block imports dependencies, defines a seed, and prints version information. The dataset block creates the DataFrame and the exact CSV copy. The perceptron block defines helpers and trains the two required binary comparisons. The least-squares block then solves the same pairwise problems using the pseudoinverse.

The logistic core is then defined once and reused in two multicategory strategies. One-vs-rest trains four class-against-all models and applies them to the training samples and a synthetic feature grid. One-vs-one trains six pairwise models and aggregates their outputs by majority vote. The final blocks serialize summaries, create an artifact manifest, optionally copy a standalone source file, build a ZIP archive, and attempt to trigger the Colab download utility.

The dependency graph is therefore simple and does not require external orchestration. A fresh Python process can run the script from top to bottom, provided the required packages are installed and the working directory is writable.

11 Design Decisions and Rationale

Several design choices are worth noting.

First, the assignment dataset is kept in the source rather than retrieved dynamically. This improves reproducibility and avoids accidental substitution of a different dataset.

Second, the main classifiers are implemented directly with NumPy. This keeps the underlying mathematics visible instead of hiding the learning procedures behind a high-level library. For an educational pattern-recognition assignment, this makes the link between the equations and the executable operations easy to follow.

Third, the binary class encoding is kept consistent within each experiment family. The perceptron and least-squares methods use ± 1 labels, while logistic regression uses 0/1 labels. This is mathematically appropriate because the perceptron update is expressed naturally in signed-label form, whereas the binary cross-entropy objective is conventionally written with 0/1 targets.

Fourth, the program makes its ambiguity rules explicit in the multiclass sections. One-vs-rest uses two thresholds to flag potentially ambiguous grid points, while one-vs-one records vote ties. Neither mechanism is a universal statistical confidence guarantee; both are operational definitions used by the implementation.

12 Computational Considerations

Let N denote the number of observations, d the number of original features, and K the number of classes. Here $d = 2$ and $K = 4$, but the dominant operations can still be described generically.

Perceptron training is approximately $O(ENd)$, where E is the number of epochs actually executed, because each sample requires a small dot product and may trigger a vector update. Least-squares training is dominated by the pseudoinverse computation and is roughly cubic in the smaller matrix dimension under conventional dense linear-algebra algorithms; the exact cost depends on the SVD-based implementation used by the numerical library.

A logistic gradient iteration is $O(Nd)$ for a single binary classifier because the main work is the matrix-vector product and the corresponding gradient matrix-vector operation. One-vs-rest

trains K logistic models, giving approximately $O(KINd)$ for I gradient iterations per model. One-vs-one trains $K(K-1)/2$ pairwise models, each on a subset of the data, so the cost grows quadratically in the number of classes at the model-count level.

The grid-based ambiguity analysis introduces an additional computational term. With G grid points, one-vs-rest prediction requires roughly $O(GKd)$ work, while one-vs-one prediction requires roughly $O(GK^2d)$ pairwise score evaluations. The implementation uses a fixed grid resolution, so this cost is predictable for the supplied dataset.

Memory usage is modest because the dataset is tiny. The largest transient arrays are the mesh grid, the flattened grid matrix, the class-score matrices, and the ambiguity masks. The source does not implement streaming, batching, or out-of-core processing because the problem size does not require them.

13 Reproducibility and Reliability

The implementation includes several features that support reproducibility: an exact in-source dataset, numerical package version reporting, deterministic configuration, explicit hyperparameters, structured CSV artifacts, a summary file, and a ZIP export mechanism. Together, these make the run easy to inspect and the training table recoverable without an external data source.

The final export workflow is also designed for Colab. After creating the ZIP file, the code attempts to use `google.colab.files.download`; outside Colab, the `ImportError` branch reports that the archive remains on disk.

The final cell also has an artifact-ordering issue. The manifest is assembled and written before the optional standalone `assignment3_solution.py` copy is performed. When that source file exists, it can therefore be included in the ZIP archive without appearing in the manifest. The archive and manifest can then disagree about the complete artifact set. This affects reproducibility documentation, but not the learning algorithms themselves.

Another reliability point concerns the use of the current working directory as the output root. Reproducing the program in a different launch location changes where artifacts are written, although the generated filenames remain stable.

14 Limitations and Technical Considerations

The source has several meaningful limitations that follow directly from the implementation.

14.1 No probabilistically normalized multiclass logistic model

The one-vs-rest approach trains four independent binary logistic models and selects the largest score. Since the four sigmoid outputs are not constrained to sum to one, the resulting score vector is not a normalized multiclass probability distribution. A dedicated multinomial logistic formulation would model the multiclass probability simplex directly, but that is not what the source implements.

14.2 Pairwise tie-breaking

One-vs-one voting explicitly detects ties but still returns an `argmax`-based class label. The returned label is therefore a deterministic tie-break rather than evidence that one class has strictly more support. Any downstream interpretation should consult the accompanying tie flag.

14.3 Logistic stopping criterion

The optimizer stops when the Euclidean norm of the proposed parameter step falls below the tolerance. This is a parameter-change criterion, not a direct gradient-norm criterion and not a formal proof of proximity to a unique optimum. In a non-convex problem this distinction could matter substantially; for logistic regression the objective is convex in $\mathbf{w}$, but the finite-optimum issue on separable data still remains.

14.4 Stored logistic loss

As noted earlier, the value recorded as `final_loss` is evaluated using the probabilities associated with the pre-update parameter vector. The returned weight vector can therefore correspond to a slightly different loss. A future refinement could recompute the loss after assigning the accepted candidate if the reporting field is intended to represent the returned model exactly.

14.5 Hyperparameter dependence

The source chooses a logistic learning rate, iteration budget, and tolerance because the assignment does not specify them. These are valid implementation choices, but they affect optimization behavior. The results of the continuous optimizer should therefore be understood as behavior under the stated deterministic configuration, not as an algorithm-independent property of the dataset.

14.6 Scope of validation

The program computes training-set diagnostics and ambiguity regions, but the source does not define a separate validation or test set. Consequently, its structured accuracy fields characterize the supplied training data rather than out-of-sample generalization.

15 Overall Technical Assessment

The implementation is technically coherent and more complete than a minimal classroom solution. It exposes the core mathematics of perceptron, least-squares, and logistic discrimination directly in NumPy, uses a common affine representation across methods, and provides reusable helpers for binary and multiclass composition. It also includes sound software-engineering practices such as explicit configuration, deterministic setup, structured serialization, and an export pipeline.

From a research-software perspective, one of the strongest aspects is the separation between algorithmic logic and artifact/reporting logic. The learning routines return structured information instead of only printing messages, which makes them easier to reuse and audit. The treatment of separable logistic problems is also explicit: non-convergence is reported rather than hidden behind a success message.

The main technical caveats concern interpretation rather than architecture. The one-vs-rest score vector is not a normalized multiclass probability distribution, the one-vs-one class label in a tie is a deterministic selection rather than a unique winner, and the reported logistic loss has a specific pre-update meaning. The final manifest ordering should also be adjusted if a complete artifact inventory is required.

Overall, the project shows a clear understanding of classical pattern-recognition methods, a direct mapping from mathematical formulations to executable NumPy operations, and attention to numerical and reproducibility issues beyond the minimum required for the assignment.

16 Conclusion

The supplied program implements a complete classical pattern-recognition workflow around four classes in a two-dimensional feature space. It constructs a deterministic dataset, trains perceptron and least-squares binary classifiers, builds binary logistic discriminants, composes them through one-vs-rest and one-vs-one multiclass schemes, evaluates ambiguity using explicit operational rules, and serializes the resulting artifacts.

The most important conceptual structure is the shared affine representation $s(\mathbf{x}) = \mathbf{w}^\top \tilde{\mathbf{x}}$. The methods differ mainly in how $\mathbf{w}$ is learned and how the resulting scores are converted into decisions: the perceptron uses signed mistake-driven updates, least squares minimizes squared residuals, and logistic regression minimizes binary cross-entropy through gradient descent. The multiclass methods then impose different decision-composition rules on those binary models.

The implementation is useful both as an assignment solution and as a compact example of how statistical learning theory, numerical linear algebra, optimization, software structure, and reproducibility practices can be combined in a transparent scientific program. Its limitations are concrete, and the code exposes enough detail for a technically informed reader to verify the relationship between the mathematical model and the actual computation.

A Source-to-Report Traceability Map

The following map records the principal source regions used in the analysis. The line ranges refer to the supplied 721-line Python file.

Table 3: Principal source regions and their roles.

Source region	Lines	Role described in this report
Environment and dataset	33–117	Imports, deterministic setup, output directory, exact dataset creation and serialization.
Perceptron	120–221	Bias augmentation, pair selection, online updates, convergence accounting, boundary generation.
Least squares	225–260	Pseudoinverse solution and binary score thresholding.
Logistic core	262–308	Sigmoid stability, cross-entropy, batch gradient descent, stopping rule.
One-vs-rest	311–448	Four binary models, argmax prediction, probability-based ambiguity analysis.
One-vs-one	450–595	Six pairwise models, probability orientation, majority voting, tie detection.
Summary and export	597–721	Summary CSV, manifest, optional source copy, ZIP archive, optional Colab download.

B Representative Code Excerpts

Only short excerpts are reproduced here, chosen because they directly expose the mathematical core of the implementation.

B.1 Perceptron update

```
if yi * np.dot(w, xi) <= 0:
    w += learning_rate * yi * xi
    mistakes += 1
```

This is the executable form of the signed-margin condition and perceptron update presented earlier.

B.2 Least-squares solution

```
w = np.linalg.pinv(X_aug) @ y_pm
scores = X_aug @ w
predictions = np.where(scores >= 0.0, 1.0, -1.0)
```

The excerpt shows the direct transition from the pseudoinverse solution to score-based classification.

B.3 Logistic gradient

```
p = sigmoid(X_aug @ w)
grad = (X_aug.T @ (p - y01)) / len(y01)
candidate = w - learning_rate * grad
```

This is the core batch gradient-descent step corresponding to the mathematical gradient derivation.